

HireReady — Study Guide for Interviews

Full-stack AI resume copilot. Upload a resume, get an ATS score against a job description, see matched/missing skills, then practice tailored interview questions with AI coaching.

Architecture Overview

Three distinct layers:

```
Browser (React + TypeScript)
  ↳ HTTP / SSE
FastAPI Backend (Python)
  ↳ API calls
External Services (Groq, Supabase, ChromaDB)
```

The core user flow:

1. Upload a resume → backend parses it, splits into chunks, embeds into ChromaDB (local vector DB)
2. Paste a job description → backend compares resume chunks against JD via Groq LLM, returns ATS score + skill gaps
3. Prep for the interview → backend generates tailored questions from resume + JD, gives coaching feedback
4. Chat with the bot → floating chatbot that has your resume as context via ChromaDB RAG

Three external services do all the heavy lifting:

- **Groq** — runs LLMs (text generation) and Whisper (speech-to-text). Fast custom LPU hardware, cheap.
- **Supabase** — two separate jobs: (1) auth (who you are via JWT), (2) storing resume files + analysis history
- **ChromaDB** — a local vector database on the server's disk. Stores resume chunks as searchable vectors so the chatbot and interview prep can find relevant content fast without re-reading the whole document

The core architectural principle: Each external dependency is isolated to exactly one service file. Swap ChromaDB for Pinecone → only `embedder_service.py` changes. Swap Groq for OpenAI → only `llm_service.py` and `interview_service.py` change. Swap Supabase Storage for S3 → only `resume_service.py` changes. Route handlers never know which provider is running.

Tech Stack

Layer	Technology	Purpose
Frontend	React + TypeScript	UI, routing, state management
Build tool	Vite	Dev server + bundler
Routing	React Router v7	Client-side navigation (Home, Interview, Profile)
State	React Context	Persist resume/analysis state across pages
HTTP client	axios	Frontend → backend requests
Auth	Supabase	User accounts, JWT sessions, hosted PostgreSQL + file storage
Backend	FastAPI (Python)	API server, business logic
Validation	Pydantic	Request/response type enforcement
PDF parser	pdfplumber	Extracts text from PDF files
DOCX parser	python-docx	Extracts text from Word files
Embeddings	sentence-transformers	Local model — converts text → vectors
Vector store	ChromaDB	Persists and searches resume chunk embeddings
LLM	Groq (llama-3.3-70b)	ATS scoring, question generation, feedback, chatbot streaming
Transcription	Groq (whisper-large-v3)	Voice answer recording → text in interview prep

Folder Structure

```
backend/app/
  api/
    routes.py          ← Wires all sub-routers together
    resume.py          ← /upload, /analyze, /history (GET+DELETE), /history/{id}/score endpoints
    interview.py       ← /questions, /feedback, /transcribe endpoints
    chat.py            ← /chat SSE streaming endpoint
    github_auth.py     ← GitHub OAuth connect/callback (origin-aware HMAC state)
  services/
    resume_service.py  ← Orchestrates: validate → upload to Supabase Storage → parse via BytesIO → chunk → embed
    parser_service.py  ← Extracts text from PDF/DOCX via BytesIO (no disk writes)
    chunker_service.py ← Splits text into overlapping 200-word windows
    embedder_service.py ← Embeds chunks + stores/queries ChromaDB; delete_chunks()
    llm_service.py     ← Groq: ATS scoring + is_valid_job_description() (shared validator)
    interview_service.py ← Groq: question generation, feedback, Whisper transcription
    mock_interview_service.py ← Agent loop: in-memory sessions, Groq tool calling, debrief generation
    chat_service.py    ← Groq: streaming chatbot with resume RAG context
    history_service.py ← Supabase history: save (INSERT), fetch (grouped), delete, delete_analysis_entry
    jd_fetcher_service.py ← Fetches JD from URL: direct HTTP first, Jina Reader fallback
    github_service.py  ← GitHub API calls (profile, repos)
  models/
    resume.py          ← Pydantic models for upload/analyze/history (includes qualification_gaps)
    interview.py       ← Pydantic models for questions/feedback
    mock_interview.py  ← Pydantic models: StartSessionRequest, SubmitAnswerRequest
    chat.py            ← Pydantic models: ChatMessage, ChatRequest

frontend/src/
  context/
    ResumeContext.tsx  ← Global state: parseResult, analyzeResult, jd, qualification_gaps
    AuthContext.tsx    ← Auth state: user, session, signIn, signUp, signOut
  lib/
    supabase.ts        ← Supabase client singleton (reads from .env)
  components/
    ResumeUpload.tsx   ← Upload widget + JD input + ATS results + qualification gaps
    AuthGate.tsx       ← Sign-in/sign-up page shown at route /
    HowItWorks.tsx     ← Interactive 4-step tutorial section
    Logo.tsx           ← SVG logo component
    ChatBot.tsx        ← Floating chat widget (SSE streaming, RAG resume context)
  pages/
    InterviewPage.tsx  ← Mock Interview (agent) + Question Bank (behavioral + role-specific)
    ProfilePage.tsx    ← Account info + sign out (sign-in form if guest)
    HistoryPage.tsx    ← Resume history: accordion by file, nested analyses, per-score delete
    App.tsx            ← Router, ResumeProvider keyed on user ID, nav, score guide, ChatBot

backend/app/core/
  config.py            ← Env vars (SUPABASE_URL, SUPABASE_ANON_KEY, GITHUB_CLIENT_*)
  supabase.py          ← Supabase client singleton
  auth.py              ← FastAPI dependencies: get_current_user, require_user
```

The key rule: route handlers are thin — they receive the request, call a service, return the response. All logic lives in services. Swap S3 for local disk → change only `resume_service.py`. Swap Pinecone for ChromaDB → change only `embedder_service.py`. Swap OpenAI for Groq → change only `llm_service.py`.

Frontend Architecture: Pages vs Components vs Context

Think of it in terms of **scope**:

Pages — "what am I looking at right now?"

A page is the full screen for a given URL. When you navigate to `/interview`, React renders `InterviewPage.tsx`. When you're at `/history`, it renders `HistoryPage.tsx`. Pages are **associated with a URL** — one per route. They own the layout of everything visible on that screen and orchestrate smaller pieces.

Important: pages do not own routing logic. They are passive — they don't know what URL they're at and don't control navigation. All routing logic lives in `App.tsx`:

```
<Routes>
  <Route path="/home" element={<HomePage />} />
  <Route path="/interview" element={<InterviewPage />} />
  <Route path="/history" element={<HistoryPage />} />
</Routes>
```

`App.tsx` says "when the URL is `/interview`, render `InterviewPage`." `InterviewPage` has no idea it's associated with `/interview` — it's just a component React Router happens to mount there.

Components — "a reusable piece of UI"

A component is a self-contained chunk of UI that doesn't care where it lives. `ChatBot.tsx` floats on every page — it's not tied to any one route. `ResumeUpload.tsx` is only mounted on the home route but is still a component because it's self-contained and doesn't own any routing logic.

The distinction: **pages compose components**. A page says "put the upload widget here, put the score guide below it." The component just knows how to render itself.

What "mounting" and "unmounting" mean

Mounting = React creates the component and puts it on screen. Think of it like opening an app on your phone — it starts up, allocates memory, displays itself.

Unmounting = React removes the component from the screen and destroys it entirely. Like closing that app — it's gone from memory.

Any `useState` variable lives **inside** the component. The moment the component unmounts, those variables are destroyed. This is why navigation loses state:

```
User is on /home → fileId = "abc-123" (lives inside HomePage)
User navigates to /interview
  → React unmounts HomePage ← fileId destroyed
  → React mounts InterviewPage ← starts fresh, knows nothing
```

Context — "state that needs to survive navigation"

Context is a **global store that lives outside any individual page**. Both `AuthContext` and `ResumeContext` wrap the entire app in `App.tsx`, so their state persists across every navigation.

```
<ResumeContext>
  {" "}
  ← never unmounts, always alive
  <Routes>
    <Route path="/home" element={<HomePage />} />
    <Route path="/interview" element={<InterviewPage />} />
  </Routes>
</ResumeContext>
```

Pages come and go. The context wrapping them never does. `file_id` stored in context survives navigation — it's not inside any page, it's above all of them.

The analogy: writing a note on a sticky note inside a book (gone when you close the book) vs writing it on the wall above the bookshelf (always there no matter which book you open).

```
App.tsx (mounts AuthContext + ResumeContext)
├── /          → ResumeUpload   ← reads/writes ResumeContext
├── /interview → InterviewPage ← reads file_id + JD from ResumeContext
├── /history   → HistoryPage
├── /profile   → ProfilePage
└── ChatBot    ← reads from both ResumeContext + AuthContext
```

`ResumeContext` is keyed on `user?.id ?? 'logged-out'` in `App.tsx`. When you log out and a different user logs in, React sees a new key and remounts the provider with fresh empty state — the next user can never see the previous user's resume data.

Backend Architecture: Routes vs Services

The analogy: Routes are waiters, services are kitchen stations.

- **Routes (waiters)** take your order (the HTTP request), hand it to the kitchen, and bring back the food (the response). They don't cook anything.
- **Services (kitchen stations)** each know how to do one thing well.

A route handler looks like this:

```
@router.post("/analyze")
async def analyze_resume(request: AnalyzeRequest, user=Depends(get_current_user)):
    result = await llm_service.analyze(request.file_id, request.job_description)
    await history_service.save(user.id, result)
    return result
```

It's just: receive → call service → return. No Groq calls, no SQL, no business logic.

FastAPI's dependency injection powers `get_current_user`. Adding `user=Depends(get_current_user)` to a route makes FastAPI automatically run that function before the handler. If the JWT is invalid, it returns a 401 and your handler never runs. It's a clean way to enforce auth without repeating the check in every route.

Why this separation matters:

- You can test each service independently without running a full HTTP server
- You can swap any provider (LLM, vector DB, file storage) by changing exactly one file
- Routes stay readable — you can understand the API surface without knowing implementation details

Pydantic Models

Pydantic is a Python library that lets you define the **shape of data** as a class and automatically validates that incoming data matches that shape before your code ever runs.

Without Pydantic you'd write validation manually:

```
if "file_id" not in request:
    raise ValueError("missing file_id")
if not isinstance(request["job_description"], str):
    raise ValueError("wrong type")
```

With Pydantic you just define:

```
class AnalyzeRequest(BaseModel):
    file_id: str
    job_description: str
```

FastAPI reads the type hint on the route handler (`body: AnalyzeRequest`), automatically parses the incoming JSON into that model, and rejects malformed requests with a 422 before your function runs. You write zero parsing or validation code.

The three model files

`models/resume.py` — shapes for the upload/analyze/history feature:

- What an `/upload` response looks like (`{ file_id, filename, word_count, sections }`)
- What an `/analyze` request needs (`{ file_id, job_description }`)
- What an `/analyze` response looks like (`{ score, matched_skills, missing_skills, qualification_gaps, summary }`)
- What a history entry looks like (`AnalysisEntry` , `ResumeFile`)

`models/interview.py` — shapes for interview prep:

- What `/questions` needs (`{ file_id?, job_description }`)
- What `/feedback` needs (`{ question, user_answer, file_id? }`)
- What a question looks like in the response (`{ question, category, hint }`)

`models/chat.py` — shapes for the chatbot:

- `ChatMessage` — one message (`{ role: "user"|"assistant", content: str }`)
- `ChatRequest` — what `/chat` receives (`{ messages: ChatMessage[], file_id?, job_description? }`)

They're separated by feature area so that adding a field to the analyze response only touches `models/resume.py` — no other model file is affected.

The `/analyze` Endpoint — Line by Line

This is the most important endpoint in the app. Here's every part:

The decorator

```
@router.post("/analyze", response_model=AnalyzeResponse)
```

Registers this function as the handler for `POST /api/resume/analyze`. `router` is mounted in `routes.py` under `/api/resume`, so the full URL is `/api/resume/analyze`. `response_model=AnalyzeResponse` tells FastAPI to validate the return value matches `AnalyzeResponse` before serializing to JSON.

The function signature

```
async def analyze(body: AnalyzeRequest, user: Optional[dict] = Depends(get_current_user)):
```

- `async def` — non-blocking. While waiting on Groq or ChromaDB, the server handles other requests instead of freezing.
- `body: AnalyzeRequest` — FastAPI sees this type hint and automatically parses the incoming JSON into the Pydantic model. Missing fields → 422 before the function runs.
- `Depends(get_current_user)` — FastAPI runs `get_current_user` first, which extracts the JWT from the Authorization header and verifies it with Supabase. The result is passed in as `user`. Invalid token → 401, function never runs. `Optional` means guests (no token) still get through — they just won't have history saved.

Gate 1 — word count

```
jd_words = body.job_description.strip().split()
if len(jd_words) < 20:
    raise HTTPException(status_code=422, detail="Job description is too short...")
```

Cheap check, no external calls. Fewer than 20 words → immediate 422.

Gate 2 — semantic validation

```
if not is_valid_job_description(body.job_description):
    raise HTTPException(status_code=422, detail="This doesn't look like a real job description...")
```

Calls `llm_service.is_valid_job_description()` — a tiny Groq call with `max_tokens=3` that asks "is this a real JD? yes or no." Catches multi-word gibberish that passes the word count. If `False` → 422, analysis never happens.

ChromaDB lookup

```
chunks = query_resume(body.file_id, body.job_description)
if not chunks:
    raise HTTPException(status_code=404, detail="No resume data found for this file_id.")
```

Calls `embedder_service.query_resume()`. Converts the JD to a vector, does a cosine similarity search in ChromaDB filtered to this `file_id`, returns the top 5 most relevant resume text chunks.

GitHub context (optional)

```
github_context = None
if user:
    connection = get_github_connection(user["id"])
    if connection:
        github_context = await fetch_github_profile(connection["github_username"], connection["github_token"])
elif body.github_username and GITHUB_TOKEN:
    github_context = await fetch_github_profile(body.github_username, GITHUB_TOKEN)
```

Two paths: logged-in users use their OAuth-connected GitHub token; guests can manually type a username and the server uses its own PAT. `github_context` is either a dict of repo data or `None`. Passed to `analyze_resume` — if `None`, the GitHub section is omitted from the prompt entirely.

The main LLM call

```
result = analyze_resume(chunks, body.job_description, github_context)
result["github_enriched"] = github_context is not None
```

Calls `llm_service.analyze_resume()` — the actual Groq call. Returns `{ score, matched_skills, missing_skills, qualification_gaps, summary }` as a plain Python dict. Tags it with `github_enriched` so the frontend knows whether GitHub data factored in.

Save to history

```
if user:
    save_analysis_result(body.file_id, result["score"], ...)
```

Only for logged-in users. Calls `history_service.save_analysis_result()` → INSERTs a new row into Supabase's `resume_history` table. Guests skip this.

Return

```
return AnalyzeResponse(**result)
```

`**result` unpacks the dict into keyword arguments. FastAPI validates it against `AnalyzeResponse`, serializes to JSON, sends back with 200.

Full call chain

```
ResumeUpload.tsx
→ POST /api/resume/analyze { file_id, job_description }
→ get_current_user()      validates JWT → user dict or None
→ word count check        no external calls
→ is_valid_job_description() fast Groq yes/no
→ query_resume()          ChromaDB cosine similarity search
→ fetch_github_profile()   GitHub API (optional)
→ analyze_resume()         main Groq call → score + skills
→ save_analysis_result()   Supabase INSERT (logged-in only)
← AnalyzeResponse JSON
→ written into ResumeContext
→ score ring + skills rendered in UI
```

How Cosine Similarity Works in the Code

Cosine similarity is not written manually — it's configured once and then handled automatically by ChromaDB.

Configuration (`embedder_service.py`)

```
_collection = client.get_or_create_collection(
    name="resumes",
    metadata={"hnsw:space": "cosine"},
)
```

`"hnsw:space": "cosine"` tells ChromaDB to use cosine similarity when searching this collection. HNSW (Hierarchical Navigable Small World) is the indexing algorithm that makes search fast — it doesn't compare your query against every stored vector one by one, it navigates a graph structure to find nearest neighbours efficiently.

On upload — storing vectors

```
embeddings = _get_model().encode(chunks, show_progress_bar=False).tolist()
_get_collection().add(
    documents=chunks,
    embeddings=embeddings,
    ids=[f"file_id_{i}" for i in range(len(chunks))],
    metadatas=[{"file_id": file_id, "filename": filename, "chunk_index": i} ...]
)
```

Each resume chunk becomes a 384-number vector stored alongside the raw text and metadata. The `file_id` in metadata is how every query stays scoped to one user's resume.

On analyze — searching vectors

```
def query_resume(file_id: str, query: str, n_results: int = 5) -> list[str]:
    query_embedding = _get_model().encode([query]).tolist()
    results = collection.query(
        query_embeddings=query_embedding,
        n_results=n_results,
        where={"file_id": file_id},
    )
    return results["documents"][0]
```

The JD text is embedded using the **same model** as the resume chunks. This is critical — if you used different models, the vectors would live in different mathematical spaces and comparison would be meaningless. ChromaDB computes cosine similarity between the JD vector and every stored chunk vector for that `file_id`, returns the 5 closest.

The full picture

```
On upload:
  resume chunks → all-MiniLM-L6-v2 → 384-dim vectors → stored in ChromaDB

On analyze:
  job description → all-MiniLM-L6-v2 → 384-dim vector
                        ↓
                ChromaDB cosine similarity search
                (angle between JD vector and each chunk vector)
                        ↓
                top 5 most similar chunk texts returned
                        ↓
                passed to Groq for scoring
```

Why the same model matters: Cosine similarity measures the angle between two vectors. That angle only means something if both vectors were produced by the same model with the same understanding of language. Same model = same vector space = valid comparison.

End-to-End Flow 1: Resume Upload

```
User drops PDF/DOCX → clicks "Upload & Analyze"
  |
  ▼ POST /api/resume/upload (multipart/form-data)
[resume_service.py]
1. Validate content type – PDF or DOCX only (400 if not)
2. Read all bytes into memory, enforce 5 MB limit (400 if over)
3. Generate UUID → upload bytes to Supabase Storage bucket "resumes"
   (no file ever written to disk)
  |
  ▼
[parser_service.py]
4. Pass bytes as BytesIO to pdfplumber (PDF) or python-docx (DOCX) → plain text
5. Regex scan each line for section keywords → detected sections list
  |
  ▼
[chunker_service.py]
6. Split text into 200-word overlapping windows (40-word overlap)
   Why overlap? A sentence crossing a boundary appears in both chunks
   so it's never missed during retrieval
  |
  ▼
[embedder_service.py]
7. all-MiniLM-L6-v2 (local, ~90MB) encodes each chunk → 384-dim vector
8. Store in ChromaDB: { text, vector, file_id, chunk_index }
  |
  ▼
Return { file_id, filename, word_count, sections } → frontend
```

Frontend saves the response into `ResumeContext` — a React Context mounted above the router. Navigating to Interview Prep and back doesn't lose the data because the context lives outside the page components.

An upload row is also INSERTed into `resume_history` (score=null) to anchor the `file_id` for history grouping.

End-to-End Flow 2: ATS Analysis

```
User pastes job description → clicks "Analyze Match"
|
▼ POST /api/resume/analyze { file_id, job_description }
[resume.py route]
- < 20 words → 422 immediately (no LLM call)
- ≥ 20 words → llm_service.is_valid_job_description() – fast yes/no Groq call
- Not a real JD → 422 "This doesn't look like a real job description"
|
▼
[embedder_service.query_resume()]
- Embed the JD using the same all-MiniLM-L6-v2 model
- ChromaDB cosine similarity search, filtered to this file_id
- Returns top-k most semantically similar resume chunks
|
▼
[llm_service.analyze_resume(chunks, job_description)]
- Sends chunks + JD to Groq (llama-3.3-70b-versatile)
- Scores across TWO dimensions: skills/keywords AND stated qualifications
  (years of experience, degree requirements, certifications)
- Returns JSON: { score, matched_skills, missing_skills, qualification_gaps, summary }
|
▼
Return AnalyzeResponse → frontend shows:
- Animated score ring (color: green ≥70, amber ≥45, red <45)
- Matched skills (green badges)
- Missing skills (red badges)
- Qualification gaps (amber warning box – stated JD requirements not met)
- Plain-English summary
- "What does this mean?" link → score guide section
- "Prep for this interview" button → Interview Prep page
```

Why INSERT instead of UPDATE for history? Each analysis is a new row so users can score the same resume against multiple job descriptions and see all results in History. Updating in-place would overwrite the previous score.

Why RAG instead of pasting the whole resume? The JD is embedded into a vector and compared against resume chunk vectors. Only the *most relevant* parts of the resume go into the LLM prompt — lower cost, better focus, and it scales to long documents.

End-to-End Flow 3: Interview Prep

Two modes. One always works without a JD, one is personalised.

Behavioral Practice

Questions are **hardcoded on the frontend** — 15 questions across 5 categories (Self, Conflict, Leadership, Growth, Motivation). No API call, instant load. User expands a question, types an answer, requests AI feedback.

Role-Specific

```
User clicks "Prep for this interview" after ATS analysis
  | (React Router passes { file_id, job_description } as state)
  ▼ POST /api/interview/questions { file_id, job_description }
[interview.py route]
- < 20 words → 422 immediately
- is_valid_job_description() → 422 if gibberish (error shown on submit, not while typing)
  |
  ▼
[embedder_service.query_resume(file_id, job_description)]
- Same ChromaDB lookup – pulls resume chunks most relevant to the JD
  |
  ▼
[interview_service.generate_questions(chunks, job_description)]
- Sends chunks + JD to Groq
- Prompt requests 8 questions: 4 behavioral (role/culture tailored)
  + 4 technical (referencing specific tools/skills from JD)
- Returns JSON array of { question, category, hint }
  |
  ▼
Frontend renders question cards (All view or One-by-one view)
```

Voice Answer Recording

```
User clicks mic button (Record) next to textarea
  |
  ▼ MediaRecorder.getUserMedia({ audio: true })
- auto-picks audio/mp4 (Safari) or audio/webm; codecs=opus (Chrome)
- button turns red + pulsing while recording
  |
User clicks Stop
  |
  ▼ POST /api/interview/transcribe (multipart audio file)
[interview_service.transcribe_audio(bytes, filename, content_type)]
- Calls Groq whisper-large-v3
- Returns { text }
  |
  ▼
Transcript appended to existing answer text in the textarea
```

AI Feedback on Answers

```
User types/records answer → clicks "Get AI Feedback"
  |
  ▼ POST /api/interview/feedback { question, user_answer, file_id }
[embedder_service.query_resume(file_id, question)]
- Embeds the question to find relevant resume chunks
  |
  ▼
[interview_service.get_feedback(question, answer, chunks)]
- Prompt to Groq is strict: if answer < 15 words or a non-answer
  ("don't know"), respond with one direct sentence – no hollow praise
- For real answers: evaluate STAR structure, specificity, measurable
  outcomes; reference specific resume experiences the candidate missed
  |
  ▼
Feedback displayed below the textarea
```

End-to-End Flow 4: Mock Interview Agent

The mock interview is a **single agent** — an LLM that controls what happens next by calling tools, rather than your code deciding the flow in advance.

```
User clicks "Begin Mock Interview"
|
▼ POST /api/mock-interview/start { job_description, file_id? }
[mock_interview_service.py]
- generate_questions() → 8 questions → take 3 behavioral + 2 technical (interleaved)
- store session in _sessions dict keyed by UUID:
  { questions, question_index: 0, history: [], current_turn, status: "active" }
- return { session_id, question: {text, category, hint}, question_number: 1, total: 5 }
|
▼ Frontend shows question 1, user types + submits answer
|
▼ POST /api/mock-interview/answer { session_id, answer }
[mock_interview_service.process_answer()]
- Look up session → get current_turn
- If primary_answer is None → this is the first answer → call _agent_decide()
- If primary_answer is set → this is a follow-up answer → always advance

[_agent_decide() - the agent loop]
Tools available:
  ask_followup(followup: str) - probe the answer with a targeted question
  advance_to_next()           - answer was sufficient, move on
  end_interview()             - all questions done

Call Groq with tool_choice="required" (forces a tool call, never free text)
|
▼ Groq returns one tool call

If ask_followup:
  → store followup question in current_turn
  → return { type: "followup", followup: "..." }

If advance_to_next / end_interview:
  → archive current_turn to history[]
  → question_index++
  → if more questions: load next → return { type: "next_question", ... }
  → if done: call _generate_debrief() → return { type: "debrief", ... }

[_generate_debrief()]
- Format full transcript (all Q+A including follow-ups)
- Single Groq call → returns JSON:
  { overall_score, hire_recommendation, overall_assessment,
    strengths[], improvements[], per_question[{question, score, feedback}] }
|
▼
Frontend renders debrief: score block, hire badge, chips, per-question accordion
```

Why tool_choice="required"? Without it, the LLM might return a plain text response instead of calling a tool. `required` makes the response shape deterministic — you always get exactly one tool call back, which maps cleanly to the three possible actions.

Session storage: In-memory Python dict — no database. Sessions are lost on server restart, which is fine for a dev app. Production would use Redis.

Max one follow-up per question: Tracked in `current_turn["followup_question"]`. Once set, the next answer is treated as the follow-up answer and always advances — the agent is never called again for that question.

End-to-End Flow 5: Job Ranker (parallel multi-agent) and Job Recon (sequential multi-agent)

Two different multi-agent patterns, both consumed from the home upload card / Job Ranker result cards.

Job Ranker — fan-out:

```
User pastes 2-5 job listing URLs, clicks Rank
|
▼ POST /api/resume/rank-jobs { file_id?, urls }
[job_ranker_service.stream_rankings()]
tasks = [asyncio.create_task(_score_url(file_id, url)) for url in urls]
for future in asyncio.as_completed(tasks):
    yield await future
    | (each _score_url independently: fetch JD → query_resume RAG → analyze_resume via Groq)
    ▼ SSE, one event per URL as it finishes — NOT in the order submitted
Frontend (JobRanker.tsx): a result card renders the instant its SSE event arrives
score ring, matched/missing chips, "Job Recon" / "Prep Interview" / "Track" buttons
```

Job Recon — sequential pipeline (LangGraph):

```
User clicks "Job Recon" on a result card (or runs it directly with a pasted JD)
|
▼ POST /api/app-intel/run { job_description, file_id? }
[app_intel_service.stream_pipeline()]
StateGraph: START → researcher → optimizer → strategist → END
researcher: JD → {company_name, tech_stack, culture_signals, key_themes, role_context}
optimizer: researcher's output + resume_chunks → 4 {original, improved, reason} bullet rewrites
strategist: researcher's output + resume_chunks → 5 {question, why_theyll_ask, category}
    | _graph.astream() yields (node_name, output) as each Groq call finishes
    ▼ SSE: one event per completed node, in strict order (can't reorder — each depends on the last)
Frontend (AppIntelPage.tsx): renders the research card, then bullet rewrites, then interview questions,
each appearing as its own step completes rather than waiting for all three
```

Why these two use different orchestration primitives: Job Ranker's per-URL scoring jobs share nothing — plain `asyncio` is enough. Job Recon's three steps share a mutating state object and strict ordering, which is exactly what LangGraph's `StateGraph` is built for. Reaching for LangGraph on the fan-out case would add ceremony with no benefit; hand-rolling the pipeline case would mean re-implementing what `StateGraph` already gives for free (typed state, step-by-step streaming via `astream()`).

End-to-End Flow 6: Chatbot

Floating chat widget visible on all pages once authed. Auto-injects resume + JD context when available.

```

User opens chat panel → types message → hits Enter
    |
    ▼ POST /api/chat/ { messages, file_id?, job_description? }
[chat.py route]
- if file_id present: query_resume(file_id, last_user_message) → top resume chunks
    |
    ▼
[chat_service.stream_chat(messages, resume_chunks, job_description)]
- Builds system prompt:
  "You are HireReady's career AI..."
  + [if chunks] USER'S RESUME: {chunks}
  + [if jd] TARGET JOB DESCRIPTION: {jd[:1500]}
  + [if no resume] direct user to upload/select rather than paste in chat
- Calls Groq with stream=True
- Yields each token as it arrives
    |
    ▼ StreamingResponse (text/event-stream)
data: {"token": "Here"}\n\n
data: {"token": " are"}\n\n
...
data: [DONE]\n\n
    |
    ▼ Frontend ReadableStream
- Each token appended to the last assistant message in state
- Auto-scrolls to bottom

```

Key design: The chatbot has context only when a resume is loaded in the current session (uploaded or selected via "Use previous resume"). Clicking "Use previous resume" sets `parseResult.file_id` in `ResumeContext`, which the chatbot picks up automatically — previous resumes are readable, they just need to be selected first.

End-to-End Flow 7: Resume History

```

User navigates to /history
    |
    ▼ GET /api/resume/history (Authorization: Bearer <JWT>)
[history_service.get_user_history(user_id)]
- Fetches all rows from resume_history for this user, ordered by uploaded_at asc
- Groups by file_id in Python:
  upload rows (score=null) → set filename + uploaded_at for the group
  analysis rows (score!=null) → append to analyses[] list
- Returns list sorted newest-first; analyses within each file sorted newest-first
    |
    ▼
Frontend renders accordion:
- One card per file_id with filename, upload date, latest score badge
- Click to expand → all ATS analyses for that resume
- Each analysis: score badge, date, "View summary" toggle, skill chips, "Prep interview"
- Trash icon → inline confirmation → DELETE /api/resume/history/{file_id}
    |
    ▼ DELETE /api/resume/history/{file_id}
[history_service.delete_resume_record(user_id, file_id)]
- Deletes all Supabase rows where user_id + file_id match
[embedder_service.delete_chunks(file_id)]
- collection.get(where={"file_id": file_id}) → finds all chunk IDs
- collection.delete(ids=...) → removes vectors

```

Why delete ChromaDB chunks on history delete? The `file_id` is orphaned once the Supabase rows are gone — vectors would accumulate forever. `delete_chunks` is in `embedder_service.py` so the architectural rule holds: one service owns one storage concern.

End-to-End Flow 8: GitHub OAuth

```
User clicks "Connect GitHub" on home upload card
|
▼ GET /api/github/connect (Authorization: Bearer <JWT>)
[github_auth.py]
- Reads Origin header from request (e.g. http://127.0.0.1:5173)
- Validates against FRONTEND_URLS whitelist
- payload = "{user_id}|{origin}"
- state = base64(payload) + "." + HMAC-SHA256(payload)[:24]
- Redirects browser to GitHub authorize URL with state param
|
▼ User authorizes → GitHub redirects to:
GET /api/github/callback?code=...&state=...
[github_auth.py]
- Decodes base64, verifies HMAC signature
- Extracts user_id + origin from payload
- Exchanges code for GitHub access token
- Saves token to Supabase
- Redirects to {origin}/home?github=connected
```

Why encode origin in state? `localhost` and `127.0.0.1` are separate localStorage origins in browsers. If the callback hardcodes `localhost` but the user came from `127.0.0.1`, the Supabase session is invisible and they see a login page. Embedding the actual `Origin` header into the HMAC-signed state fixes this permanently.

End-to-End Flow 9: Application Tracker

```
"Track this job" clicked on ATS results card / Job Ranker card
|
▼ AddApplicationModal pre-filled with whatever's already known
  (company_name, job_title, job_url, file_id, score, jd_snippet)
▼ POST /api/applications/ { company_name, job_title, job_url?, file_id?, score?, jd_snippet?, notes? }
[application_service.create_application()]
row = { user_id, status: "saved", **data }
INSERT into job_applications, return the created row
|
▼ User navigates to /tracker
▼ GET /api/applications/ → list_applications(), ordered by updated_at desc
TrackerPage.tsx renders 5 columns (Saved/Applied/Interviewing/Offer/Rejected),
filtering the flat list into each column by `status`
|
▼ User drags a card to a new column (native HTML5 drag-and-drop – no DnD library)
onDragStart: dataTransfer.setData('text/plain', app.id)
onDrop (target column): moveStatus(id, newStatus)
|
▼ Optimistic update FIRST – setApplications(...) locally, THEN:
▼ PATCH /api/applications/{id} { status }
[application_service.update_application()]
UPDATE job_applications SET status=..., updated_at=now() WHERE id=... AND user_id=...
| if the PATCH fails → catch block calls load() again → re-fetches truth from the server,
| silently reverting the optimistic change (rollback-via-refetch, not a manual undo)
```

Why optimistic update + rollback-via-refetch instead of waiting for the server response? Drag-and-drop feels broken if the card snaps back into place for a network round-trip before moving. The UI updates instantly on drop; the PATCH happens in the background. On the rare failure, refetching the whole list is simpler and more correct than trying to hand-roll a targeted undo of one optimistic mutation.

Why `job_applications` has its own `file_id` / `score` / `jd_snippet` columns instead of just a foreign key to `resume_analyses` : applications can be tracked from three different entry points — ATS results, Job Ranker results, or a manual "Add application" button with none of that context. Denormalizing the snapshot values onto the tracker row means a tracked application still shows a score even after the original resume or analysis is deleted from history.

Supabase Schema

The original design used one flat `resume_history` table (upload rows with `score=null` + analysis rows in the same table, grouped by `file_id` in Python). That was migrated to two properly related tables, plus a third for the Application Tracker:

```
-- resume_files (one row per upload)
file_id      text
user_id      uuid
filename     text
uploaded_at  timestampz

-- resume_analyses (one row per ATS analysis, FK'd to resume_files.file_id)
file_id      text
score        int
matched_skills jsonb
missing_skills jsonb
jd_snippet   text (first 300 chars of the JD)
summary      text (Grog-generated plain-English summary)
analyzed_at  timestampz

-- job_applications (application tracker)
id           uuid primary key default gen_random_uuid()
user_id      uuid
company_name text
job_title    text
job_url      text (nullable)
status       text (saved | applied | interviewing | offer | rejected)
file_id      text (nullable – resume used, if tracked from an ATS/ranker flow)
score        int (nullable – ATS score at time of tracking)
jd_snippet   text (nullable)
notes        text (nullable)
created_at   timestampz
updated_at   timestampz
```

`get_user_history` does a real Supabase **join**, not Python grouping — `.select("file_id, filename, uploaded_at, resume_analyses(score, matched_skills, missing_skills, jd_snippet, summary, analyzed_at)")` — so an upload with zero analyses still returns a row with an empty `analyses` list, and the frontend never sees blank cards. Splitting into two tables (instead of the original single flat table) means each upload row is written exactly once instead of being duplicated across every analysis, and a `resume_files` row with no analyses yet is representable without a null-score sentinel row.

No RLS on any table — access control is enforced in application code via the Supabase admin/service-role client with explicit `.eq("user_id", ...)` filters on every query.

Embeddings: Local vs Cloud

An embedding is a list of numbers representing the *meaning* of text. Two sentences with similar meaning → vectors that are close together, even if they share no words.

```
"5 years of Python experience"      → [0.12, -0.87, 0.34, ...]
"built Python microservices since 2019" → [0.13, -0.85, 0.31, ...] ← close!
```

	Local (sentence-transformers)	Cloud (OpenAI text-embedding-3-small)
Cost	Free	~\$0.00002/1K tokens
Dimensions	384	1536
Privacy	Data stays on server	Data leaves your machine
Setup	Download once (~90MB)	API key required
Swap effort	One line in <code>embedder_service.py</code>	One line in <code>embedder_service.py</code>

ChromaDB

Stores vectors so they can be searched later. Lives at `backend/chroma_db/` — SQLite-backed, persists across restarts.

Each chunk stored as:

- `id` : `{file_id}_{chunk_index}`
- `document` : raw text
- `embedding` : 384 floats
- `metadata` : `{ file_id, filename, chunk_index }`

The `where={"file_id": file_id}` filter in every query ensures you only search the current user's resume — not everyone's. Similarity is measured by **cosine similarity** (angle between vectors — smaller angle = more similar meaning).

Groq vs Other LLM Options

Provider	Model	Cost	Key needed?
Groq	llama-3.3-70b	Free tier	Yes (free)
OpenAI	gpt-4o-mini	~\$0.001/req	Yes (paid)
Anthropic	claude-haiku-4-5	~\$0.001/req	Yes (paid)
Ollama	llama3 (local)	Free	No

Groq is used now because it's free and fast. Switching to OpenAI or Anthropic for production is a single import change in `llm_service.py` and `interview_service.py` — nothing else in the codebase is aware of which provider is running.

Note: `llama3-8b-8192` was decommissioned by Groq mid-project. Always check console.groq.com/docs/deprecations if you get a 400 `model_decommissioned` error.

Authentication Flow

Supabase is a hosted backend-as-a-service built on PostgreSQL. It provides auth, a relational database, and file storage — all from one service. For auth, it runs its own Auth server alongside your database.

```

User submits email + password on AuthGate (route /)
  |
  ▼ supabase.auth.signInWithPassword()
Supabase Auth server
- Checks credentials against internal auth.users table
- Returns a JWT (JSON Web Token) — a signed string encoding
  user ID, email, and expiry
- Supabase JS client stores JWT in localStorage automatically
  |
  ▼
AuthContext.onAuthStateChange fires
- Sets user + session in React state
- axios.defaults.headers.common['Authorization'] = `Bearer <token>`
- All future API calls now carry the token automatically
  |
  ▼
React Router sees user is non-null → redirects from / to /home
  |
  ▼ Any protected API call (e.g. POST /api/resume/upload)
FastAPI — get_current_user dependency (auth.py)
- Extracts Bearer token from Authorization header
- Calls supabase.auth.get_user(token) — Supabase verifies signature
- Returns { id, email } if valid; raises 401 if forged/expired

```

Why the JWT is safe to pass around: The token is cryptographically signed by Supabase using a secret key only they hold. Your backend never sees the user's password — only the signed token. If anyone tampers with it, the signature check fails and it's rejected.

Session refresh: JWTs expire after 1 hour. The Supabase JS client silently refreshes them using a refresh token (also in localStorage). `onAuthStateChange` fires on each refresh and updates the axios header.

Guest mode: "Continue without signing in" stores a `guestMode` flag in `sessionStorage`. The app treats this as authenticated for routing purposes. Signing out clears the flag and returns to the auth gate.

Backend auth dependencies (auth.py):

- `get_current_user` — optional auth. Returns user dict or `None`. Existing endpoints stay backward-compatible.
- `require_user` — throws 401 if no valid token. Used when an endpoint must be locked to signed-in users.

Supabase Storage

Files are stored in Supabase Storage (a hosted S3-compatible object store) — no local disk involved. On upload, bytes are read into memory and uploaded directly to the `resumes` bucket with the UUID as the key. `parser_service.py` receives bytes and opens them via `BytesIO`, so `pdfplumber` and `python-docx` never touch disk.

```

Supabase Storage bucket: resumes/<uuid>.pdf (or .docx)
ChromaDB:                file_id → embedded chunks
resume_history table:    file_id + user_id → scores, metadata

```

The `file_id` is the key that ties all three together. Deleting a resume means: delete the Supabase history rows, delete the ChromaDB chunks. The storage object itself is retained (could be cleaned up with a Supabase storage delete call if needed).

The next step for production would be adding a proper `files` table with `user_id` as a foreign key, so the app can list a user's uploaded files without relying on `resume_history` upload rows.

Interview Talking Points

"Walk me through a user request end to end."

The user uploads a PDF. React sends it as multipart/form-data to `POST /api/resume/upload`. The service layer validates type and size, saves with a UUID filename, extracts text using pdfplumber, splits into 200-word overlapping chunks, embeds each chunk with a local sentence-transformers model, and stores the vectors in ChromaDB tagged with a `file_id`. When the user pastes a job description, we embed that too, query ChromaDB for the most semantically similar resume chunks, and send them to Groq to produce an ATS score and gap analysis. The same `file_id` and JD then flow to the interview prep page, where Groq generates role-specific questions grounded in both the resume and the JD.

"What is RAG and why does this project use it?"

RAG is Retrieval-Augmented Generation — instead of sending a full document to the LLM every time, you index it once as vectors and retrieve only the relevant parts at query time. Here, the resume is chunked and embedded on upload. At analysis time, the job description is embedded and the closest resume chunks are retrieved via cosine similarity. Only those chunks go into the Groq prompt. This keeps costs low, avoids context window limits, and lets the LLM focus on the most relevant content.

"Why local embeddings instead of OpenAI?"

Local embeddings with sentence-transformers cost nothing and keep user data on the server. The model downloads once (~90 MB) and runs in-process as a lazy singleton. Quality is very good for semantic similarity on short resume chunks. Switching to OpenAI's text-embedding-3-small in production is one line in `embedder_service.py` — nothing else in the codebase knows which model produced the vectors.

"How does state persist when the user navigates between pages?"

React Router unmounts page components on navigation, which normally destroys local state. We store the persistent data — parse result, ATS analysis, job description — in a React Context (`ResumeContext`) mounted above the router in the component tree. Context survives navigation because it's never unmounted. Ephemeral state like drag/drop and error messages stays local to the component since it doesn't need to persist.

"How does authentication work in this app?"

Authentication is handled by Supabase. On sign-in, Supabase's auth server verifies the credentials and returns a JWT. The Supabase JS client stores the JWT in `localStorage` and auto-refreshes it before expiry. Our `AuthContext` listens for auth state changes and sets the JWT as the default `Authorization` header on every axios request — so all API calls carry it automatically without any component needing to think about it. On the backend, a FastAPI dependency (`get_current_user` in `auth.py`) extracts the token and calls `supabase.auth.get_user()` to verify the signature. If the token is forged or expired, Supabase rejects it and we return a 401. The current endpoints support optional auth — they still work without a token — but `require_user` is available to lock down endpoints that must be tied to an account. When the user logs out, `ResumeProvider` is keyed on `user?.id ?? 'logged-out'` so it remounts with fresh empty state — the next login can't accidentally see the previous user's resume data.

"How does the resume history work, and why INSERT instead of UPDATE?"

History is stored in a Supabase table called `resume_history`. There are two row types: upload rows (score=null, created on file upload) and analysis rows (score set, inserted on each ATS run). We INSERT new rows instead of updating because a user might analyze the same resume against five different job descriptions — UPDATE would silently overwrite every previous score. The `get_user_history` service groups rows by `file_id` in Python so the frontend sees one accordion card per resume with a nested list of all scores. Deleting a resume removes all Supabase rows for that `file_id` and calls `delete_chunks()` in `embedder_service.py` to clean up the ChromaDB vectors so they don't accumulate as orphans.

"How does GitHub OAuth work and what was tricky about it?"

It's a standard three-leg OAuth flow — redirect to GitHub, get a code back, exchange for a token. The tricky part was the post-callback redirect. In development the frontend runs on both `localhost:5173` and `127.0.0.1:5173`, which browsers treat as completely separate localStorage origins. If you hardcode the redirect to `localhost` but the user came from `127.0.0.1`, the Supabase session is invisible after the redirect and they see a login page. The fix: read the `Origin` header from the `/connect` request, encode it with the user ID as `user_id|origin`, sign the payload with HMAC-SHA256 using the GitHub client secret, and pass it as the OAuth state parameter. The callback decodes and verifies it, then redirects to whichever origin is in the payload — so localhost and 127.0.0.1 both work correctly without any hardcoded URLs.

"How does the chatbot work and what context does it have?"

The chatbot is a floating widget that streams responses token-by-token using Server-Sent Events. When the user sends a message, the frontend POSTs to `/api/chat/` with the conversation history plus the current `file_id` and `job_description` from React Context. The backend queries ChromaDB with the last user message to retrieve the most relevant resume chunks, then builds a system prompt with those chunks and the JD. That prompt goes to Groq with `stream=True` — each token is yielded as a `data: {"token": "..."} SSE event. The frontend reads the stream with a ReadableStream reader and appends each token to the last message in state, giving a real-time typing effect. If no resume is loaded, the system prompt instructs the LLM to direct the user to upload or select one — it never asks users to paste their resume in the chat window.`

"How does voice recording work across Safari and Chrome?"

The Web Speech API only works in Chrome/Edge, so we use `MediaRecorder` + Groq Whisper instead, which works in all modern browsers. The `VoiceMicButton` component calls `getUserMedia({ audio: true })` for mic permission, then starts a `MediaRecorder`. It detects the best supported MIME type — `audio/webm;codecs=opus` for Chrome, `audio/mp4` for Safari — using `MediaRecorder.isTypeSupported()`. On stop, it assembles the recorded chunks into a Blob and POSTs it as multipart form data to `/api/interview/transcribe`. The backend strips the codec suffix from the content-type header, calls Groq's `whisper-large-v3` model, and returns the transcript. The transcript is appended to whatever the user has already typed, so typing and speaking can be mixed freely.

"How do you prevent garbage input from getting an ATS score or interview questions?"

Two layers. First, a word count check — fewer than 20 words gets rejected immediately before any LLM call. Second, a semantic check: a fast Groq call with `max_tokens=3` asks "is this a real job description?" and returns yes/no. This costs almost nothing but catches multi-word gibberish that passes the word count. Both checks live in shared code — `is_valid_job_description()` in `llm_service.py` — so `/analyze` and `/questions` use the exact same validation without duplication. On the frontend, the error only appears when the user clicks submit, not while they're typing — this avoids distracting mid-composition warnings.

"What's the difference between RAG and an agent? Which does this app use?"

RAG is a retrieval pattern — your code retrieves context, stuffs it into a prompt, calls the LLM once, and returns. The LLM is passive. An agent is different: the LLM actively decides what to do next by calling tools. The mock interview uses an agent loop. After each candidate answer, the LLM picks one of three tools — `ask_followup`, `advance_to_next`, or `end_interview`. We use Groq's tool calling with `tool_choice="required"` to force a tool call every time, which makes the response shape predictable. The session state (transcript, current question index) lives server-side between HTTP requests. So each `/answer` call is one iteration of the agent loop — the LLM sees the question and answer, reasons about quality, and decides the next action without your code prescribing it.

"How is the mock interview agent different from just calling Groq and checking the output?"

A naive approach would be: call Groq, parse the text response to see if it says "follow up" or "move on", branch on that. That's fragile — text parsing breaks, the LLM might say something unexpected, and you have no schema enforcement. Tool calling is different: you define structured tools with typed parameters, and the LLM returns a JSON tool call object instead of free text. `tool_choice="required"` ensures you always get a tool call back. The response is always one of three known shapes — no text parsing, no branching on strings. It's the difference between asking someone a yes/no question in prose versus giving them two buttons to click.

"Walk me through the two multi-agent flows you built — Job Ranker and Job Recon."

Both are multi-agent, but they're different orchestration patterns because the sub-tasks have different data dependencies. Job Ranker is fan-out: the user pastes 2-5 job URLs, and I fire one scoring coroutine per URL with `asyncio.create_task` + `asyncio.as_completed` — each one fetches its own JD, does the same RAG-and-Groq scoring as `/analyze`, and results stream back over SSE the instant each one finishes, not in submission order. That's safe because the URLs are completely independent — no agent needs another's output. Job Recon is the opposite: a researcher agent extracts company/tech-stack/culture signals from the JD, a resume optimizer rewrites bullets using the researcher's output, and an interview strategist generates company-specific questions using both prior outputs. That has to be sequential — the optimizer's prompt literally can't be built until the researcher's JSON comes back. I used LangGraph for that one specifically: a `StateGraph` with a shared typed state each node reads and writes, and `astream()` gives me step-by-step SSE events without hand-rolling a generator.

"Why LangGraph for Job Recon but plain asyncio for Job Ranker? Why not use LangGraph everywhere?"

LangGraph earns its keep when there's a state machine to model — nodes that depend on shared, evolving state and need explicit ordering. Job Recon is exactly that. Job Ranker has no shared state at all — each URL's scoring is a completely isolated coroutine — so `asyncio.gather` / `as_completed` is simpler and has zero framework overhead. Reaching for LangGraph on independent parallel work would just be ceremony; hand-rolling the pipeline's ordering and state-passing in Job Recon would mean re-implementing what `StateGraph` already gives for free.

"How does the Application Tracker's drag-and-drop work, and what happens if the backend call fails?"

Native HTML5 drag-and-drop — no external library. `onDragStart` stashes the application ID in `dataTransfer`; the target column's `onDrop` reads it and calls `moveStatus`. The status update is optimistic: React state updates immediately so the card visually moves before any network round-trip, then a `PATCH /api/applications/{id}` fires in the background. If that PATCH fails, the catch block just re-fetches the full list from the server — that's the rollback. It's simpler than hand-rolling a targeted undo, and for a five-column kanban board a full re-fetch is cheap enough that the simplicity wins.

"How would you scale this for real users?"

A lot of this is already done, not hypothetical — resume files already go to Supabase Storage rather than local disk, and Supabase's Postgres tables already map every `file_id` to a `user_id`. The remaining swaps are still single-file by design: ChromaDB moves to Pinecone or pgvector (one file — `embedder_service.py`) once a single server's local disk stops being enough; Groq swaps to OpenAI or Anthropic for production SLAs (two files — the LLM services); mock interview session state moves from an in-memory Python dict to Redis so sessions survive a server restart or load-balancing across multiple instances. Supabase's Row Level Security could also replace the current pattern of enforcing `user_id` filters in application code, pushing that guarantee down to the database itself.

"Why FastAPI over Flask or Django?"

FastAPI has async support out of the box — critical for LLM API calls that take several seconds. Pydantic handles request validation automatically (wrong shape → 422, no manual checks). Auto-generated OpenAPI docs at `/docs`. Django is overkill for an API-first backend; Flask requires wiring Pydantic and async manually.

"What is CORS and why do you need it?"

CORS is a browser policy that blocks requests to a different origin than the page was loaded from. Frontend is on port 5173, backend on 8000 — different origins. FastAPI's `CORSMiddleware` adds `Access-Control-Allow-Origin` headers so the browser permits the cross-origin requests.

AI Agents — Single and Multi-Agent

What makes something an "agent" vs a regular LLM call?

In every endpoint built before the mock interview, **your code** decides what happens: call ChromaDB, call Groq, return the result. The LLM is a text transformer — it answers one question and stops.

An **agent** flips the control. The LLM decides what to do next by calling tools. Your code defines the available tools and runs a loop:

```
while True:
    response = LLM(messages, tools)
    if response is a tool call:
        result = execute_tool(response.tool_name, response.args)
        messages.append(result)      ← feed result back
    else:
        return response.text        ← LLM decided it's done
```

The LLM accumulates context across iterations and decides when to stop. That's the core idea.

Single Agent

One LLM in a loop with a set of tools. The mock interview is this pattern:

```
User submits answer
↓
Groq (tools: ask_followup | advance_to_next | end_interview)
↓
Picks one tool → backend executes it → returns result to frontend
```

Each HTTP request is one iteration of the loop. The loop state (session) lives server-side between requests. The LLM sees the question and the candidate's answer, then picks the right tool — it's not told which one to use, it reasons about the answer quality and decides.

Key properties:

- One LLM, multiple tool calls over time
- State accumulates between iterations (full transcript builds up)
- `tool_choice="required"` makes the response shape predictable — always a tool call, never free text
- The agent can ask at most one follow-up per question (enforced in session logic, not by the LLM)

Multi-Agent

Multiple agents coordinated by an **orchestrator**. Each sub-agent has a narrow job and its own tools. Two patterns:

Parallel (fan-out) — built: Job Ranker

All sub-agents start at the same time. Total time = slowest agent, not sum. This is shipped, not hypothetical — `job_ranker_service.py`.

```
POST /api/resume/rank-jobs { file_id?, urls: [2..5 JD URLs] }
|
▼
stream_rankings(file_id, urls)
  tasks = [asyncio.create_task(_score_url(file_id, url)) for url in urls]
  for future in asyncio.as_completed(tasks):
    yield await future          ← yields each result AS it finishes, not in URL order
    |
    ▼ each _score_url() coroutine, independently:
    1. fetch_jd_from_url(url)    ← reuses jd_fetcher_service (Ashby fast path, Jina fallback)
    2. reject if JD < 50 words
    3. query_resume(file_id, jd_text) ← ChromaDB RAG, same as /analyze
    4. analyze_resume(chunks, jd_text) ← same Groq scoring call as /analyze
    |
    ▼ StreamingResponse, one SSE event per completed URL
    data: {"url":..., "ok": true, "score": 82, "matched_skills": [...], ...}
    data: {"url":..., "ok": false, "error": "..."} ← per-URL failure doesn't kill the batch
    ...
    data: [DONE]
```

Why `asyncio.as_completed` instead of `asyncio.gather`: `gather` waits for every task and returns one list at the end — the user stares at a blank screen until the slowest JD (often a slow-loading page) finishes. `as_completed` yields each result the moment it's ready, so a card renders the instant its scoring finishes, independent of the others. Total wall-clock is still bounded by the slowest URL, but perceived latency is much lower.

Why each `_score_url` call is wrapped in its own `try/except`: One bad URL (dead link, login-walled page, sparse content) returns `{ok: false, error: ...}` instead of raising and killing the other 4 in-flight scoring calls.

Only works because the jobs are **independent** — no agent needs another's output to start.

Sequential (pipeline) — built: Job Recon

Each agent's output feeds the next. Can't parallelize because of data dependencies. Implemented with **LangGraph** (`app_intel_service.py`), not hand-rolled — a `StateGraph` with one shared `TypedDict` state that each node reads from and writes back into.


```

POST /api/app-intel/run { job_description, file_id? }
  |
  ▼
AppIntelState = { job_description, resume_chunks, company_name, tech_stack,
                  culture_signals, key_themes, role_context,
                  bullet_suggestions, strategic_questions }

StateGraph(AppIntelState)
  START → researcher → optimizer → strategist → END

  _researcher(state) → Groq call → {company_name, tech_stack, culture_signals,
                                   key_themes, role_context}
    ↓ merged into state
  _optimizer(state) → Groq call, prompt includes researcher's fields +
                    resume_chunks → 4 {original, improved, reason} bullets
    ↓ merged into state
  _strategist(state) → Groq call, prompt includes researcher's fields +
                    resume_chunks → 5 {question, why_theyll_ask, category}

  |
  ▼
_graph.astream(initial) yields (node_name, node_output) as each node finishes
  |
  ▼
  SSE: data: {"step": "researcher", "data": {...}}
      data: {"step": "optimizer", "data": {...}}
      data: {"step": "strategist", "data": {...}}
      data: [DONE]
  |
  ▼
  AppIntelPage.tsx renders each card as its step's SSE event arrives

```

Why LangGraph instead of three sequential function calls in a for-loop? For three linear nodes, a plain function chain would work almost identically — LangGraph's value here is the explicit `StateGraph` (one typed schema every node reads/writes, so nodes can't silently drop fields) and `astream()` giving step-by-step events for free instead of hand-rolling a generator. It also sets up cleanly for future branching/looping (e.g. a "revise strategist output" edge) without restructuring the flow.

Why it can't be parallel: the optimizer's prompt literally interpolates `state["company_name"]` , `state["tech_stack"]` , etc. — fields that don't exist until the researcher node returns. Same for the strategist, which depends on both prior nodes' output.

RAG vs Agents vs Multi-Agent — the key difference

	RAG (analyze/questions/chat)	Single Agent (mock interview)	Multi-Agent (Job Ranker / Job Recon)
Who controls flow	Your code	The LLM	Orchestrator (asyncio / LangGraph) + LLMs
LLM calls per request	1	N (one per answer)	N (fan-out) or 3 (pipeline)
State between calls	None	Session dict	Shared <code>AppIntelState</code> (pipeline) / none (fan-out)
Latency	~1-2s	~2-4s per turn	Job Ranker: max(single-URL latency). Job Recon: sum of 3 sequential calls
Best for	Defined, predictable tasks	Open-ended with decision branches	Specialization (pipeline) or independent parallel work (fan-out)

MCP (Model Context Protocol)

MCP is a standard protocol for connecting to external tool servers over a consistent interface (stdio or HTTP transport, JSON-RPC underneath) instead of hand-writing a bespoke client for every API. Once connected, the caller lists the server's tools and invokes them by name with structured args — the same shape regardless of which server it is.

Built: GitHub MCP ([github_service.py](#)) — spawns [@modelcontextprotocol/server-github](#) as a subprocess over stdio:

```
server_params = StdioServerParameters(
    command="npx", args=["-y", "@modelcontextprotocol/server-github"],
    env={**os.environ, "GITHUB_PERSONAL_ACCESS_TOKEN": token},
)
async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()
        search = await session.call_tool("search_repositories", {"query": f"user:{username} sort:stars", "perPage": 8})
        ...
        readme = await session.call_tool("get_file_contents", {"owner": username, "repo": repo, "path": "README.md"})
```

The result (top non-fork repos + decoded READMEs, forks filtered out, markdown stripped) is folded into the ATS scoring prompt in [llm_service.py](#), so a candidate's actual GitHub activity can supplement — or fill gaps in — what's on the resume.

Note this is separate from GitHub OAuth ([github_auth.py](#)): OAuth is how the app gets a **token** for a given user (the three-leg redirect/callback/HMAC-state dance described below). MCP is what the app **does** with that token once it has it — call the GitHub MCP server's tools instead of hand-rolling raw GitHub REST calls. Two different concerns, wired together: OAuth produces the credential, MCP consumes it.

Planned: LinkedIn MCP — fetch JD text from LinkedIn URLs the same way. Currently blocked: LinkedIn actively blocks automated access, and no public LinkedIn MCP server exists yet — low feasibility until that changes. This is why [jd_fetcher_service.py](#) returns an explicit "can't fetch LinkedIn/Indeed" error rather than silently failing.

What's Built

Feature	Where
Resume upload + Supabase Storage	<code>POST /api/resume/upload</code> , <code>resume_service.py</code>
PDF/DOCX parsing via BytesIO (no disk writes)	<code>parser_service.py</code>
Text chunking (overlapping windows)	<code>chunker_service.py</code>
Local embeddings + ChromaDB storage	<code>embedder_service.py</code>
ATS scoring via Groq (skills + qualifications)	<code>POST /api/resume/analyze</code> , <code>llm_service.py</code>
Qualification gaps warning (amber UI)	<code>AnalyzeResponse.qualification_gaps</code> , <code>ResumeUpload.tsx</code>
Semantic JD validation (gibberish detection)	<code>llm_service.is_valid_job_description()</code> , used in <code>/analyze</code> + <code>/questions</code>
Matched/missing skills + score ring	<code>ResumeUpload.tsx</code>
Resume history (accordion, nested analyses)	<code>GET /api/resume/history</code> , <code>history_service.py</code> , <code>HistoryPage.tsx</code>
Delete full resume from history	<code>DELETE /api/resume/history/{file_id}</code> , <code>delete_chunks()</code>
Delete individual ATS score from history	<code>DELETE /api/resume/history/{file_id}/score?at=</code> , <code>delete_analysis_entry()</code>
ATS summary saved + shown in history	<code>summary</code> col in <code>resume_history</code> , <code>HistoryPage.tsx</code>
Context-aware upload card header	<code>App.tsx</code> <code>HomePage</code> reads <code>parseResult</code> / <code>analyzeResult</code>
Logo click resets state (with confirm modal)	<code>AppInner.handleLogoClick()</code> , <code>App.tsx</code>
State reset on login/logout	<code><ResumeProvider key={user?.id ?? 'logged-out'}></code>
GitHub OAuth (origin-aware HMAC state)	<code>github_auth.py</code>
JD URL fetch (direct HTTP + Jina fallback)	<code>POST /api/resume/fetch-jd</code> , <code>jd_fetcher_service.py</code>
Role-specific interview questions	<code>POST /api/interview/questions</code> , <code>interview_service.py</code>
Behavioral question bank (15 questions)	<code>InterviewPage.tsx</code> (hardcoded, 5 category groups)
AI feedback on answers	<code>POST /api/interview/feedback</code> , <code>interview_service.py</code>
Voice answer recording → Whisper transcription	<code>POST /api/interview/transcribe</code> , <code>VoiceMicButton</code> in <code>InterviewPage.tsx</code>
All / One-by-one question views	<code>InterviewPage.tsx</code>
Mock interview agent (tool calling loop)	<code>POST /api/mock-interview/start</code> + <code>/answer</code> , <code>mock_interview_service.py</code>
Agent follow-up decision (Groq tool choice)	<code>_agent_decide()</code> in <code>mock_interview_service.py</code>
Scored debrief with hire recommendation	<code>_generate_debrief()</code> in <code>mock_interview_service.py</code>
Chatbot (SSE streaming, RAG context)	<code>POST /api/chat/</code> , <code>chat_service.py</code> , <code>ChatBot.tsx</code>
Cross-page state persistence	<code>ResumeContext.tsx</code>
Nav + SVG logo + user avatar	<code>App.tsx</code> , <code>Logo.tsx</code>
ATS score guide section	<code>App.tsx</code>
Auth gate (sign in / sign up / guest)	<code>AuthGate.tsx</code> , route <code>/</code>

Feature	Where
Supabase JWT auth — frontend	<code>AuthContext.tsx</code> , <code>lib/supabase.ts</code>
Supabase JWT auth — backend	<code>core/auth.py</code> , <code>core/supabase.py</code>
Profile page (account info + sign out)	<code>ProfilePage.tsx</code> , route <code>/profile</code>
GitHub MCP profile enrichment	<code>github_service.py</code> (stdio MCP client), folded into ATS prompt
Split history storage (files vs analyses)	<code>resume_files</code> + <code>resume_analyses</code> tables, <code>history_service.py</code>
Job Ranker — parallel fan-out multi-agent	<code>POST /api/resume/rank-jobs</code> , <code>job_ranker_service.py</code> , <code>JobRanker.tsx</code>
Job Recon — sequential LangGraph pipeline	<code>POST /api/app-intel/run</code> , <code>app_intel_service.py</code> , <code>AppIntelPage.tsx</code>
Application Tracker — kanban board	<code>POST/GET/PATCH/DELETE /api/applications</code> , <code>application_service.py</code> , <code>TrackerPage.tsx</code>
Shared "Track this job" modal	<code>AddApplicationModal.tsx</code> (used by ResumeUpload, JobRanker, TrackerPage)

What's Next

Feature	Notes
PostgreSQL files table	Proper <code>files</code> table with <code>user_id</code> FK for listing uploads outside <code>resume_files</code> / <code>resume_analyses</code>
LinkedIn MCP	Fetch real JD text from LinkedIn URLs — blocked on LinkedIn's anti-scraping posture, no public MCP server exists yet
Email-parsing automation	Stretch goal on the Application Tracker — auto-detect status changes (interview invites, rejections, offers) from a connected inbox

Architectural Tradeoffs

These are the decisions worth being able to explain and defend.

1. ChromaDB (local) vs Cloud Vector DB (Pinecone, Weaviate)

What it is: ChromaDB runs on the same machine as FastAPI. It's SQLite-backed — just files on disk at `backend/chroma_db/` . No network calls, no accounts needed.

	ChromaDB (local)	Pinecone/Weaviate (cloud)
Cost	Free	\$70+/month at scale
Latency	~5ms (same machine)	~50–200ms (network hop)
Setup	<code>pip install chromadb</code>	API keys, account, config
Persistence	Survives server restarts	Survives server restarts + crashes
Serverless compatible	No	Yes
Scale	Single server only	Horizontally scalable

The real risk — serverless deployments (e.g. Vercel):

A normal server is always running with persistent disk access. A serverless platform like Vercel spins up a fresh container per request and throws it away after. There is no persistent filesystem. So:

```
Request 1 → Container A → embeddings written to /tmp/chroma_db ✓
Request 2 → Container B (fresh) → /tmp/chroma_db doesn't exist ✗
```

This has nothing to do with logging out. Logging out is just a browser action — it clears a JWT. The problem is that between any two requests, Vercel might give you a completely new container. ChromaDB assumes a persistent filesystem; serverless doesn't provide one.

For production: Replace ChromaDB with Pinecone or Supabase's pgvector extension. Only `embedder_service.py` changes — nothing else in the codebase knows which vector store is running.

2. Supabase Storage (cloud) vs Local Disk for Resume Files

What it is: Raw PDF/DOCX bytes go directly to Supabase's `resumes` bucket on upload. The server never writes a file to its own disk.

	Supabase Storage	Local Disk
Survives server restart	Yes	No
Multi-server deployments	Yes (shared)	No (each server has its own copy)
Cost	Generous free tier	Free (your disk)
Complexity	Needs bucket config	<code>open(path, 'wb')</code>
Privacy	Third party stores the files	Files stay on your server

Why `BytesIO` matters: `parser_service.py` takes the raw bytes and opens them with `BytesIO` — an in-memory file-like object. Neither `pdfplumber` nor `python-docx` ever see a real file path. The bytes come in over HTTP, go to Supabase and to the parser simultaneously in memory, and nothing hits disk. This makes the backend stateless — restart it, and nothing is lost because no state ever lived on its disk.

3. Stateless JWT vs Server-Side Sessions

What it is: JWTs are self-contained signed tokens. The server verifies them cryptographically without looking up anything in a database.

	JWT (stateless)	Server sessions
Server memory needed	Zero	Grows with active users
Instant revocation	Hard (token lives until expiry)	Easy (delete session row)
Horizontal scaling	Trivial (any server can verify)	Needs shared session store (Redis)
Complexity	Supabase handles it entirely	You manage session storage

The tradeoff most people miss: You can't instantly log someone out with JWTs. If a token has a 1-hour expiry and you want to forcibly revoke it, the token is still valid until it expires — you can't "un-sign" it. Supabase mitigates this with short expiry (1 hour) + silent background refresh using a separate refresh token. For this app it's a non-issue.

Why stateless matters for scaling: With a normal session, every server needs to be able to look up "is session ID abc123 valid?" — which means either one central database (bottleneck) or a shared cache like Redis. With JWTs, any server that has the signing key can verify any token independently. No coordination needed.

4. Groq vs OpenAI

	Groq	OpenAI
Speed	Very fast (custom LPU hardware)	Moderate
Cost	Free tier, very cheap	More expensive
Model quality	Good (Llama 3.3 70B)	GPT-4o is stronger
Reliability	Smaller company, newer	Industry standard SLA
Voice (Whisper)	Same model, much cheaper	Same model, pricier

Groq's LPU (Language Processing Unit): Custom silicon designed specifically for LLM inference — not repurposed GPU hardware. Makes streaming feel noticeably faster. For a demo, the difference is visible.

Why this decision is low-risk: All Groq calls are isolated to `llm_service.py` and `interview_service.py`. Switching to OpenAI is changing two import lines and two API call formats — nothing in routes, context, or the frontend knows which LLM is running.

5. Local Embeddings vs API Embeddings

	Local (sentence-transformers)	OpenAI text-embedding-3-small
Cost	Free after one-time download	~\$0.00002/1K tokens
Privacy	Resume text never leaves server	Sent to OpenAI
Cold start	~2–3s first request (model loads)	Instant
Vector size	384 dimensions	1536 dimensions
Quality	Good for resume matching	Slightly better

Why privacy matters here: Resumes contain personal information — names, addresses, employment history. With local embeddings, that text only goes to Groq (for scoring) and Supabase (for storage, explicitly consented to). It never goes to a third-party embeddings API. That's a selling point for enterprise use.

The cold start: The `all-MiniLM-L6-v2` model is loaded as a lazy singleton — it loads on the first embedding request after startup and stays in memory. First request after a cold start takes 2–3 seconds. Every subsequent request is fast.

6. RAG vs Sending the Whole Resume

What it is: RAG (Retrieval-Augmented Generation) = index once as vectors, retrieve only relevant parts at query time, send only those to the LLM.

Alternative: Paste the entire resume text into every LLM prompt.

	RAG (this app)	Full document in every prompt
Cost per request	Low (only relevant chunks)	High (full resume every time)
Context window limit	Not a concern	Breaks for long resumes
Focus	LLM sees only relevant content	LLM must process everything
Upfront work	Must chunk + embed on upload	Nothing on upload
Latency	Extra ChromaDB lookup	Simpler, slightly faster

For this use case RAG is the right call: A resume is 500–1500 words. The JD is another 500–1000. Together they approach or exceed context limits for some models. More importantly, RAG makes the LLM focus — when scoring a "Python engineering" JD, you want the LLM to see your Python projects, not your education section.

Demo Script

Lead with what the app does for the user. Explain the architecture only when asked.

Opening (30 seconds)

"Most people apply to jobs without knowing if their resume will get past the ATS filter — the automated system that screens resumes before a human ever sees them. This app tells you your score, what's missing, and then helps you prep for the interview."

Step 1 — Upload

Drag the resume in. While it processes:

"The backend is parsing the PDF in memory using pdfplumber, splitting the text into overlapping chunks, and embedding each chunk with a local ML model. Those embeddings go into ChromaDB — a local vector database. This is what lets the chatbot and interview prep semantically search your resume later without re-reading the whole document every time."

Step 2 — Analyze

Paste a JD, click Analyze. While it processes:

"It embeds the job description into a vector, finds the most semantically similar resume chunks via cosine similarity, and sends those chunks to Groq's Llama 3.3 70B model to produce a structured score."

When results appear:

"You get a score, matched skills, missing skills — and qualification gaps, which are things the JD explicitly requires that your resume doesn't address. Like '5 years required' when your resume doesn't state years."

Step 3 — Interview Prep

Click "Prep for this interview":

"Because the app already has my resume and the JD in context, questions start generating immediately. These aren't generic — Groq sees both my resume and the role requirements and generates questions grounded in what I actually wrote."

Show feedback:

"The feedback prompt explicitly bans hollow praise. If your answer is weak, it tells you why and what to add."

Step 4 — Chatbot

Open the chat widget:

"The chatbot has my resume as context. It uses the same RAG pipeline — my message gets embedded, ChromaDB returns the most relevant resume chunks, and those go into the system prompt. I can ask 'do I have any DevOps experience?' and it's searching my actual resume to answer."

Step 5 — History

Navigate to History:

"Every ATS analysis is saved to Supabase — a cloud Postgres database. I can come back days later, see all my scores across different job descriptions, and delete individual entries. Deleting a resume removes both the database rows and the ChromaDB vectors so they don't accumulate as orphans."

Common follow-up questions and tight answers

"Why Groq instead of OpenAI?"

"Groq runs on custom LPU hardware — significantly faster and cheaper than GPU-based providers. And because all LLM calls are isolated to two service files, switching providers is a one-afternoon job."

"How does the chatbot know my resume?"

"RAG pipeline. Message gets embedded, ChromaDB finds the most relevant resume chunks via cosine similarity, chunks go into the system prompt. The LLM never re-reads the whole resume — just the relevant parts."

"Could this scale?"

"The main thing to swap is ChromaDB for Pinecone or pgvector — needed because ChromaDB requires a persistent filesystem and serverless platforms don't provide one. Everything else — Supabase Storage for files, Postgres tables keyed by user_id, Groq — is already cloud-native. That swap is a single-file change by design, isolated to `embedder_service.py`."

"How does auth work?"

"Supabase issues a JWT on sign-in. The JS client stores it and auto-refreshes it before expiry. Every API call sends it in the Authorization header. The FastAPI backend verifies the signature with Supabase on each request — fully stateless, no session storage server-side."

"Why local embeddings instead of OpenAI?"

"Two reasons: cost and privacy. Resumes contain personal data — with local embeddings it never leaves the server to go to a third-party API. And it's free. Quality is sufficient for semantic similarity on short resume chunks. Swapping to OpenAI embeddings in production is one line in `embedder_service.py`."

"What is a vector embedding?"

"A list of numbers that encodes the meaning of text. Two sentences with similar meaning produce vectors that are mathematically close together, even if they share no words. 'Five years of Python experience' and 'built Python microservices since 2019' produce vectors with a small angle between them. That's what cosine similarity measures — the angle."

"Why INSERT instead of UPDATE for history rows?"

"A user might analyze the same resume against five different job descriptions. UPDATE would silently overwrite the previous score. INSERT gives you a full history. The tradeoff is the table grows, but each row is tiny — it's the right call for this use case."

Behavioral Interview Questions — STAR Answers

Use the **STAR** structure (Situation, Task, Action, Result) and keep each answer to roughly 60-90 seconds out loud. The project-grounded answers below use real decisions from this codebase — they hold up if an interviewer asks a follow-up, because they're describing what actually happened, not a story built to fit the question. The two teamwork/conflict questions at the end are flagged because this was a solo project — swap in a real example from a job, group project, or internship, using the same STAR skeleton.

"Tell me about a challenging technical problem you solved."

Situation: GitHub OAuth login worked fine when I tested it, but I noticed it silently broke depending on which URL I happened to open the app from. **Task:** I needed to figure out why the same login flow succeeded from one address and failed from another, since a flaky auth flow would be a dealbreaker for real users. **Action:** I traced it to something non-obvious: `localhost:5173` and `127.0.0.1:5173` are the same server but different browser origins, so `localStorage` — where the Supabase session lives — doesn't carry over between them. If the OAuth callback hardcoded a redirect to `localhost` but the user had started from `127.0.0.1`, the session was invisible after redirect and they'd land back on a login screen with no error message at all. I fixed it by reading the actual `Origin` header off the initial `/connect` request, encoding it into the OAuth `state` parameter alongside the user ID, and signing that payload with HMAC-SHA256 so it couldn't be tampered with. The callback decodes and verifies the signature, then redirects back to whichever exact origin the user came from. **Result:** Both `localhost` and `127.0.0.1` — and by extension any deployed domain — work correctly with zero hardcoded URLs anywhere in the flow. It also taught me to test auth flows from more than one entry point, since this kind of bug produces no error, no log line, and no crash — just a user who silently gets logged out.

"Tell me about a time you had to learn something new quickly and apply it."

Situation: I wanted to add a multi-step "Job Recon" feature — research a company from its job posting, then use that research to rewrite resume bullets, then generate interview questions — where each step genuinely depends on the last one's output. **Task:** I'd only built single-call LLM features up to that point. I needed a way to model an explicit multi-step pipeline with shared state, rather than three functions called back-to-back with parameters threaded through by hand. **Action:** I learned LangGraph specifically for this — a graph-based framework for orchestrating multi-step LLM workflows. I modeled the pipeline as a `StateGraph` with one shared, typed state object (`AppIntelState`) that each of the three nodes reads from and writes into, wired `START → researcher → optimizer → strategist → END`, and used `astream()` to get each node's output the moment it finished, which I forward to the frontend as Server-Sent Events so the UI renders each card as it's ready instead of waiting for all three calls to finish. **Result:** The pipeline works, streams incrementally, and — because I used the same shared-state pattern rather than passing individual variables between three separate functions — it's straightforward to extend with a fourth node or a conditional branch later without restructuring anything. It also gave me a clear mental model for when a full orchestration framework is worth reaching for versus when plain functions are enough (I made the opposite call for the Job Ranker feature, which is why it uses plain `asyncio` instead — see the multi-agent talking points above).

"Tell me about a tradeoff you had to make under constraints."

Situation: Every part of this app — the LLM, the embeddings, the vector store — had a "free and simple" option and a "paid and more scalable" option, and I was building it solo without production traffic yet. **Task:** I had to pick defaults that let me move fast and validate the product without accumulating cost or infrastructure I didn't need yet, but without backing myself into a corner. **Action:** I chose Groq over OpenAI (free tier, extremely fast inference on their custom LPU hardware, good-enough quality with Llama 3.3 70B), local `sentence-transformers` embeddings over an embeddings API (free, keeps resume PII on the server, one-time 90MB model download), and ChromaDB over Pinecone (zero setup, SQLite-backed, runs on the same machine). The one architectural rule I enforced throughout was that every one of these is isolated to a single service file — `llm_service.py` owns the LLM provider, `embedder_service.py` owns the vector store — so route handlers and the frontend never know which provider is running underneath. **Result:** I got a fully working product with zero infrastructure cost while building it. And because of that isolation rule, I know exactly what it costs to move to production-grade providers later: it's a one- or two-file change per swap, not a rewrite. The tradeoff I accepted knowingly is that ChromaDB's local-disk dependence means it won't work as-is on a serverless platform like Vercel — that's a known, explicit limitation, not a surprise I'd discover later.

"Tell me about a mistake you made and what you learned from it."

Situation: I initially modeled resume history as one flat Supabase table — a single row type doing double duty as both "a file was uploaded" (score left null) and "a score was recorded" (score populated), grouped into a nested structure in Python after fetching. **Task:** As I added features — deleting individual scores, showing an upload with zero analyses yet — the null-sentinel design started producing awkward edge cases, and the grouping logic in `history_service.py` was doing work that really belonged in the database. **Action:** I recognized the design was fighting me rather than helping, so I migrated to two properly related tables — `resume_files` and `resume_analyses` — with a real foreign-key join, replacing Python-side grouping with a single Supabase query using `.select("file_id, filename, uploaded_at, resume_analyses(...)")`. **Result:** Every upload row now means exactly one thing, an upload with no analyses yet just returns an empty nested list instead of needing a null-score placeholder row, and the deletion logic got simpler because there's no ambiguity about which row means what. The lesson I took from it: when I catch myself writing "if score is null, treat this row as X, otherwise treat it as Y" — that's usually a sign the data model needs two tables, not clever branching in the service layer.

"Tell me about a time you had to debug something without a clear error message."

Situation: Voice-recorded interview answers worked perfectly in Chrome during testing, but early on I knew Safari support would be flaky since browsers don't agree on audio recording formats. **Task:** I needed transcription to work across Chrome, Safari, and Firefox without the user ever noticing a difference, despite each browser producing a different audio container format from `MediaRecorder`. **Action:** Rather than picking one format and hoping, I used `MediaRecorder.isTypeSupported()` to detect the best available MIME type per browser at record time — `audio/webm;codecs=opus` for Chrome, `audio/mp4` for Safari, `audio/ogg` for Firefox — and made the backend strip the codec suffix before handing the bytes to Groq's Whisper model, which accepts all three underlying formats. **Result:** Voice answers transcribe correctly regardless of which browser a candidate happens to be using, with no separate code path per browser beyond the one feature-detection call. It reinforced a habit: for anything touching browser media APIs, assume the three major engines will disagree, and design the detection in from the start rather than patching it in after a bug report.

"Tell me about a project you're proud of and why."

Situation: I built HireReady end-to-end, solo — a resume/interview copilot that scores a resume against a job description, generates tailored interview questions, and runs an adaptive mock interview. **Task:** Beyond making individual features work, I wanted the codebase itself to hold up: swappable providers, thin route handlers, real separation between RAG, single-agent, and multi-agent patterns rather than blurring them together. **Action:** I held to one architectural rule throughout — every external dependency (LLM provider, vector store, file storage, auth) is isolated to exactly one service file, so route handlers never know which provider is running. I used that same discipline to deliberately build three different agentic patterns as the app grew: RAG for the core ATS scoring, a single Groq tool-calling agent for the adaptive mock interview, and two different multi-agent orchestration styles — fan-out with plain `asyncio` for the parallel Job Ranker, and a LangGraph `StateGraph` pipeline for the sequential Job Recon flow — picking the simplest tool that fit each problem's actual data dependencies instead of using one framework everywhere. **Result:** The app does something genuinely useful — I use it myself when applying to jobs — and the architecture means I can point to exactly why each technical decision was made, not just that it works. That's the version of "proud of" that matters to me: not just shipping features, but being able to defend every choice under questioning.

"How do you prioritize when scope is ambiguous or time is limited?"

Situation: Building solo meant every feature decision was mine to make, and there was no shortage of directions to take the product — LinkedIn integration, more agent patterns, deeper GitHub analysis. **Task:** I had to decide what to build now versus defer, without a product manager setting priorities for me. **Action:** I used a simple test: build the core RAG loop (upload → analyze → interview prep) fully first, since nothing else matters if the baseline product doesn't work well. Then I looked at what would meaningfully differentiate the product versus what was interesting-but-marginal. LinkedIn JD fetching is a good example of a deliberate no for now — I looked into it, found LinkedIn actively blocks automated access and no public MCP server exists for it, and rather than sinking time into a scraper that would break constantly, I documented it as explicitly blocked and moved on to the Application Tracker, which had a clear, buildable path and immediate user value. **Result:** The features that shipped — mock interview agent, Job Ranker, Job Recon, the tracker — all work reliably because I didn't split focus chasing a low-feasibility integration. Recognizing "this is technically interesting but low-probability of paying off soon" as a reason to deprioritize, rather than a challenge to power through, is something I actively practice.

"Tell me about a time you disagreed with an approach and changed direction."

(Framed here as disagreeing with my own earlier decision — swap in a real disagreement with a teammate or manager if you have one, using the same STAR shape.)

Situation: My first instinct for job scoring was to let a user paste one job description at a time and score it. As I used the app more myself, I realized most job searches involve comparing several postings at once, and the one-at-a-time flow didn't match how people actually job hunt. **Task:** I had to decide whether to keep pushing on making the single-JD flow better, or invest in a different flow entirely for comparing multiple jobs at once. **Action:** I built the Job Ranker as a genuinely different flow rather than bolting "compare mode" onto the existing single-JD UI — it takes 2-5 URLs, fetches and scores each one independently in parallel, and streams ranked results back as each one finishes. That meant writing a second scoring code path, but one that reuses `jd_fetcher_service` and `analyze_resume` rather than duplicating logic. **Result:** The two flows now serve different real use cases — deep analysis against one target job versus fast comparison across several — instead of one compromised flow trying to do both. The reused service functions meant the second flow only added an `asyncio` fan-out layer, not a parallel scoring implementation.

"Tell me about a time you had a conflict with a coworker or teammate."

(This project was built solo, so there's no real example here — bring a genuine one from a job, internship, or group project. Use this skeleton:

Situation — *what was the disagreement and why did it arise (different priorities, different read on requirements, different technical opinions)?*

Task — *what did you need to resolve, and by when?* **Action** — *how did you engage the other person: did you ask questions before asserting*

your view, propose a small test/prototype to settle it empirically, escalate, or compromise? **Result** — what actually got decided, and what did you learn about how you handle disagreement. Interviewers are listening for whether you engaged the disagreement directly and respectfully, not whether you "won.")

"Tell me about a time you received critical feedback."

(Also best answered with a real example — a code review, a manager's note, a professor's critique. Use the same skeleton: **Situation/Task** — what was the feedback and what was it responding to? **Action** — your first reaction (defensive instincts are normal — did you take a beat before responding?), then what you actually changed. **Result** — the outcome, and specifically what you'd do differently next time given that feedback. If you're short on examples, a legitimate one for this project: a friend or classmate might have suggested the ATS scoring prompt was too strict on entry-level roles — describe how you'd validate that by testing edge-case resumes against the prompt before deciding whether to loosen it.)

"Why does this project matter to you? / Why are you interested in this kind of work?"

Situation: Job searching is genuinely opaque from the applicant's side — you have no idea if an ATS is filtering you out, no idea what a specific company actually wants beyond the JD's buzzwords, and generic interview prep doesn't address the role you're actually interviewing for. **Task:** I wanted to build something that gives a candidate the same kind of insight a well-connected friend at the company might give them — informally: here's what they likely care about, here's how your resume reads against this specific posting, here's what they'll probably ask and why. **Action:** That's the thread running through every feature — the ATS scoring doesn't just match keywords, it also checks stated qualifications like years of experience or degree deadlines; Job Recon explicitly reverse-engineers what a company's JD implies about their priorities before generating anything; the mock interview agent probes vague answers with a follow-up instead of just moving on, the way a real interviewer would. **Result:** It's the project I keep extending because I keep finding uses for it myself, which is the best signal that it's solving a real problem and not just a portfolio exercise.